Greedy Algorithms

Week 5 – Algorithms

Korea University Sejong Campus, Department of Computer Science

Part 1. Greedy Algorithm Fundamentals

Quiz 3 (Week 04 review) was at the start of this hour.

What is a Greedy Algorithm?

A type of algorithm for solving **optimization problems**

- **Optimization problem**: Find the best (maximum or minimum) solution among all feasible solutions
- At each step, make the **locally optimal choice** – the one that looks best *right now*
- Once a choice is made, it is **never reconsidered** (no backtracking)

Generic Greedy Structure:

```
Greedy(C):                          // C = set of all candidates
    S ← {}
    while C ≠ {} and S is not a complete solution:
        x ← the best-looking element in C
        C ← C - {x}
        if S ∪ {x} is feasible:
            S ← S ∪ {x}
    if S is a complete solution: return S
    else: return "no solution"
```

Key Properties for Greedy Correctness

For a greedy algorithm to produce an **optimal** solution, two properties must hold:

1. Greedy-Choice Property

- A globally optimal solution can be reached by making locally optimal (greedy) choices
- We can prove that at least one optimal solution includes the greedy choice

2. Optimal Substructure

- An optimal solution to the problem contains optimal solutions to its subproblems
- After making a greedy choice, the remaining subproblem has the same structure

> **Greedy vs Dynamic Programming:** Both require optimal substructure. But DP considers *all* subproblems (bottom-up), while Greedy makes *one* choice and moves forward (top-down, no revisiting).

When Greedy Fails – Example 1

Binary Tree Maximum Path Sum

```
          10
        /    \
      36      15
     /  \    /  \
   3   18  35    2
  / \ / \ / \  / \
 30 45 55 50 32 67 38 33
```

- **Greedy**: At each node, go to the child with the larger value
  - Path: 10 -> 36 -> 18 -> 55 = **119**
- **Optimal**: 10 -> 15 -> 35 -> 67 = **127**

The greedy choice at the root (36 > 15) locks us out of the best path.

When Greedy Fails – Example 2

Coin Change with Non-Standard Denominations

Standard denominations (500, 100, 50, 10, 1 won):

- Each denomination is a **multiple** of the next smaller one
- Greedy always gives **optimal** result

Non-standard denominations (e.g., add 160-won coin):

- Make 200 won: Greedy picks 160 + 10 + 10 + 10 + 10 = **5 coins**
- Optimal: 100 + 100 = **2 coins**

> **Key insight**: Greedy works for coin change only when denominations have a special structure (each is a multiple of the next smaller). For arbitrary denominations, use **Dynamic Programming**.

Coin Change – Greedy Algorithm

Problem: Given denominations (500, 100, 50, 10, 1), find the minimum number of coins for amount W.

```
CoinChange(W):
    change = W
    n500 = n100 = n50 = n10 = n1 = 0
    while change ≥ 500: change -= 500; n500++
    while change ≥ 100: change -= 100; n100++
    while change ≥ 50:  change -= 50;  n50++
    while change ≥ 10:  change -= 10;  n10++
    while change ≥ 1:   change -= 1;   n1++
    return n500 + n100 + n50 + n10 + n1
```

Example: W = 760

- 500-won: 1 coin (remaining 260)
- 100-won: 2 coins (remaining 60)
- 50-won: 1 coin (remaining 10)
- 10-won: 1 coin (remaining 0)
- **Total: 5 coins** (optimal for standard denominations)

Fractional Knapsack Problem

Problem definition:

- n items, each with weight w_i and value v_i
- Knapsack capacity C
- **Fractional**: items can be divided – take any fraction of an item
- **Goal**: Maximize total value in the knapsack

Greedy idea:

1. Compute **value per unit weight** (v_i/w_i) for each item
2. Sort items by value-per-weight in **decreasing** order
3. Take items greedily – whole items if they fit, fractional otherwise

> **0-1 Knapsack**: Items cannot be divided (take it or leave it) – requires **DP** or backtracking. **Fractional Knapsack**: Items can be divided – solved optimally by **Greedy**.

Fractional Knapsack – Algorithm

```
FractionalKnapsack(items, C):
    Input:  n items (each with weight, value), capacity C
    Output: list L of items in knapsack, total value v

    1. Compute value-per-weight for each item
    2. Sort items by value-per-weight in decreasing order → S
    3. L = {}, w = 0, v = 0
    4. for each item x in S (in sorted order):
    5.     if w + x.weight ≤ C:
    6.         add x entirely to L
    7.         w += x.weight; v += x.value
    8.     else:
    9.         fraction = (C - w) / x.weight
    10.        add fraction of x to L
    11.        v += fraction * x.value
    12.        break
    13. return L, v
```

Fractional Knapsack – Worked Example

Knapsack capacity = 40 kg

Item	Weight	Value	Value/Weight
Platinum	10 kg	60	**6**
Gold	15 kg	75	**5**
Silver	25 kg	10	0.4
Tin	20 kg	2	0.1

Sorted by value/weight: Platinum, Gold, Silver, Tin

Step	Action	w	v
1	Take all Platinum (10 kg)	10	60

Part 2. Classic Greedy Algorithms

Job Scheduling

Problem: Assign n jobs (each with start time and finish time) to the **minimum number of machines** such that no two jobs on the same machine overlap.

Greedy strategy: Earliest Start Time First

```
JobScheduling(jobs):
    Input:  n jobs, each with [start, finish]
    Output: assignment of jobs to machines

    1. Sort jobs by start time → L
    2. while L is not empty:
    3.     take job t with earliest start time from L
    4.     if some existing machine is free at time t.start:
    5.         assign t to that machine
    6.     else:
    7.         create a new machine and assign t
    8.     remove t from L
    9. return machine assignments
```

> Among 4 strategies (earliest start, earliest finish, shortest first, longest first), only **earliest start time first** guarantees an optimal solution for the machine-minimization variant.

Job Scheduling – Worked Example

Jobs: t1=[7,8], t2=[3,7], t3=[1,5], t4=[5,9], t5=[0,2], t6=[6,8], t7=[1,6]

Sorted by start time: [0,2], [1,6], [1,5], [3,7], [5,9], [6,8], [7,8]

Step	Job	Assignment	Machines
1	[0,2]	Machine 1	M1: [0,2]
2	[1,6]	Machine 2 (M1 busy until 2)	M1: [0,2], M2: [1,6]
3	[1,5]	Machine 3 (M1,M2 busy)	+M3: [1,5]
4	[3,7]	Machine 1 (free at 2)	M1: [0,2],[3,7]
5	[5,9]	Machine 3 (free at 5)	M3: [1,5],[5,9]
6	[6,8]	Machine 2 (free at 6)	M2: [1,6],[6,8]

Activity Selection (Single Machine Variant)

Problem: One machine (room), maximize the **number of non-overlapping jobs**.

Greedy strategy: **Earliest Finish Time First**

```
ActivitySelection(jobs):
    Sort jobs by finish time
    selected = {first job}
    last_finish = first job's finish time
    for each remaining job j:
        if j.start ≥ last_finish:
            selected = selected ∪ {j}
            last_finish = j.finish
    return selected
```

- Among the three strategies (shortest job, earliest start, earliest finish), only **earliest finish time first** guarantees optimal for the single-machine variant.
- **Time complexity**: O(n log n)
- This is the classic **Activity Selection Problem** from CLRS Ch. 16.1

Huffman Coding – Concept

Problem: Compress a file by assigning **variable-length binary codes** to characters

Key ideas:

- Frequent characters get **shorter** codes
- Rare characters get **longer** codes
- Codes satisfy the **prefix property**: no code is a prefix of another
  - e.g., if 'a' = `101`, no other code starts with `101`, and `1` and `10` are not codes

Advantage of prefix property:

- No separator needed between codes
- Unambiguous decoding by reading bits left to right

> Huffman coding builds a **binary tree** based on character frequencies. Each leaf is a character. Left edges = 0, right edges = 1. The path from root to leaf gives the code.

Huffman Coding – Algorithm

```
HuffmanCoding(characters, frequencies):
    Input:  n characters with their frequencies
    Output: Huffman tree

    1. Create a leaf node for each character, storing its frequency
    2. Build a min-priority queue Q from all nodes (by frequency)
    3. while |Q| ≥ 2:
    4.     A = extract-min(Q)      // lowest frequency
    5.     B = extract-min(Q)      // second lowest
    6.     create new node N
    7.     N.left = A, N.right = B
    8.     N.freq = A.freq + B.freq
    9.     insert N into Q
    10. return root of Q           // root of Huffman tree
```

Time complexity: O(n log n)

- Building heap: O(n)
- Loop runs (n-1) times, each iteration: 2 extract-min + 1 insert = O(log n)
- Total: O(n) + (n-1) * O(log n) = **O(n log n)**

Huffman Coding – Worked Example

Character frequencies: A: 450, T: 90, G: 120, C: 270

Step-by-step tree construction:

```
Step 1: Q = [T:90, G:120, C:270, A:450]
        Merge T(90) + G(120) → node(210)

Step 2: Q = [210, C:270, A:450]
        Merge 210 + C(270) → node(480)

Step 3: Q = [A:450, 480]
        Merge A(450) + 480 → node(930)
```

Resulting tree and codes:

```
        930
       /   \
     A:450  480          A  = 0    (1 bit)
           /   \         C  = 10   (2 bits)
        210   C:270      T  = 110  (3 bits)
       /   \             G  = 111  (3 bits)
     T:90  G:120
```

Huffman Coding – Compression Ratio

Using the codes: A = `0`, C = `10`, T = `110`, G = `111`

Compressed file size:

Char	Freq	Code Length	Bits Used
A	450	1	450
C	270	2	540
T	90	3	270
G	120	3	360
Total	**930**		**1,620 bits**

Original (8-bit ASCII): 930 * 8 = 7,440 bits

Minimum Spanning Tree (MST)

Problem: Given a weighted, connected graph G = (V, E), find a tree T that:

- Connects **all** vertices (spanning)
- Has **minimum** total edge weight

Properties of a spanning tree:

- Exactly **n - 1** edges (where n = |V|)
- No cycles
- Adding any edge creates exactly one cycle

Two classic greedy algorithms:

	Kruskal's	Prim's
Strategy	Add cheapest edge that doesn't form a cycle	Grow tree from a start vertex, always adding the cheapest connecting edge

Kruskal's MST Algorithm

```
KruskalMST(G):
    Input:  weighted graph G = (V, E), |V| = n, |E| = m
    Output: MST T

    1. Sort all edges by weight in ascending order → L
    2. T = {}
    3. while |T| < n - 1:
    4.     e = next smallest edge from L
    5.     if adding e to T does not create a cycle:
    6.         T = T ∪ {e}
    7.     else:
    8.         discard e
    9. return T
```

Cycle detection: Use **Union-Find** data structure

- `Find(u)` and `Find(v)`: if same set, adding edge (u,v) creates a cycle
- `Union(u,v)`: merge two sets when edge is added
- With union-by-rank + path compression: nearly O(1) per operation

Time complexity: O(m log m) – dominated by sorting edges

Prim's MST Algorithm

```
PrimMST(G):
    Input:  weighted graph G = (V, E), |V| = n, |E| = m
    Output: MST T

    1. Pick arbitrary start vertex p; D[p] = 0
    2. for each vertex v ≠ p:
    3.     if edge (p, v) exists: D[v] = weight(p, v)
    4.     else: D[v] = infinity
    5. T = {p}
    6. while |T| < n:
    7.     vmin = vertex not in T with minimum D[v]
    8.     add vmin and edge (u, vmin) to T    // u is in T
    9.     for each vertex w not in T:
    10.        if weight(vmin, w) < D[w]:
    11.            D[w] = weight(vmin, w)       // update
    12. return T
```

Key difference from Kruskal: Prim grows **one tree** from a starting vertex; Kruskal merges **multiple trees** (forest).

Time complexity: O(n^2) with array, O(m log n) with binary heap

Kruskal vs Prim — Visual Comparison

Kruskal's (edge-centric — merges forests):

```
Step 1:  a--b  c  d  e  f     (add cheapest edge)
Step 2:  a--b  c--d  e  f     (add next cheapest)
Step 3:  a--b--c--d  e  f     (forests merge)
  ...    one tree gradually forms
```

Prim's (vertex-centric — grows one tree):

```
Step 1:  [c]                  (start from c)
Step 2:  [c--b]               (nearest vertex to tree)
Step 3:  [c--b--f]            (nearest vertex to tree)
  ...    tree grows outward
```

Both produce the same MST (if edge weights are unique), but the construction order differs.

Dijkstra's Shortest Path Algorithm

Problem: Find shortest paths from source vertex s to all other vertices in a weighted graph (non-negative weights).

```
Dijkstra(G, s):
    Input:  weighted graph G = (V, E), source vertex s
    Output: array D where D[v] = shortest distance from s to v

    1. D[v] = infinity for all v; D[s] = 0
    2. S = {}                       // set of finalized vertices
    3. while S ≠ V:
    4.     vmin = vertex not in S with minimum D[v]
    5.     add vmin to S            // finalize vmin
    6.     for each neighbor w of vmin not in S:
    7.         if D[vmin] + weight(vmin, w) < D[w]:
    8.             D[w] = D[vmin] + weight(vmin, w)  // edge relaxation
    9. return D
```

Dijkstra's Algorithm – Edge Relaxation

Edge relaxation is the core operation (lines 6-8):

```
s ──────→ ... ──────→ vmin ──────→ w
|         D[vmin]        wt        |
|                                  |
+──────────→ ... ──────────────→  w
             D[w] (current)
```

- If `D[vmin] + weight(vmin, w) < D[w]`, then going through vmin is a **shorter path** to w
- Update: `D[w] = D[vmin] + weight(vmin, w)`

Why it works: When vmin is selected (minimum D value among unfinalized), its shortest distance is **guaranteed correct** – no future path through unfinalized vertices can be shorter (because all edge weights are non-negative).

Time complexity: O(n^2) with array, O(m log n) with binary min-heap

> **Note**: Dijkstra's does NOT work with **negative** edge weights. For that, use **Bellman-Ford** (O(VE)).

Dijkstra's – Worked Example

Graph: Cities with distances (Seoul as source)

Step	Finalized	Action	Key Updates
0	Seoul (0)	Initialize	Neighbors get direct distances
1	Cheonan (12)	Relax from Cheonan	Update Daejeon, Wonju, etc.
2	Wonju (15)	Relax from Wonju	Update Gangneung
3	Nonsan (via Cheonan)	Relax from Nonsan	Update Gwangju, Daegu
4	Daejeon	Relax from Daejeon	No improvements
5	Daegu	Relax from Daegu	Update Busan, Pohang
…	Continue until all finalized		

Algorithm Complexity Summary

Algorithm	Problem	Time Complexity	Optimal?
Coin Change (Greedy)	Min coins	O(k) for k denominations	Only for standard denominations
Fractional Knapsack	Max value (divisible)	**O(n log n)**	Yes (always)
Job Scheduling (EST)	Min machines	O(n log n) + O(mn)	Yes (always)
Activity Selection (EFT)	Max activities, 1 machine	**O(n log n)**	Yes (always)
Huffman Coding	Optimal prefix code	**O(n log n)**	Yes (always)
Kruskal's MST	Min spanning tree	**O(m log m)**	Yes (always)

Greedy vs Dynamic Programming

	Greedy	Dynamic Programming
Choice	Make the locally best choice *now*	Consider *all* subproblems
Direction	Top-down (make choice, then solve subproblem)	Bottom-up (solve all subproblems, then combine)
Revisiting	Never reconsider a choice	Stores and reuses all subproblem solutions
Correctness	Only if greedy-choice property holds	Always correct if optimal substructure holds
Speed	Typically faster	Typically slower (but more general)
Coin Change	O(k), but may fail	O(nk), always optimal

Summary

Greedy algorithms make locally optimal choices at each step, hoping to find a global optimum.

Two conditions for correctness:

- **Greedy-choice property**: a locally optimal choice leads to a globally optimal solution
- **Optimal substructure**: optimal solution contains optimal solutions to subproblems

Core algorithms covered today:

1. **Coin Change** – greedy works for standard denominations only
2. **Fractional Knapsack** – sort by value/weight, O(n log n)
3. **Job Scheduling** – earliest start time first, O(n log n + mn)
4. **Huffman Coding** – build tree from lowest frequencies, O(n log n)
5. **Kruskal's MST** – cheapest edge that doesn't form cycle, O(m log m)
6. **Prim's MST** – grow tree by nearest vertex, O(n^2)
7. **Dijkstra's** – shortest path via edge relaxation, O(n^2)

Next Week

- **Dynamic Programming** – optimal substructure, overlapping subproblems, memoization vs tabulation

Q & A

codingchild@korea.ac.kr